

cp4101 b.comp. dissertation (final presentation)

linking rust code with lean

filbert phang
(supervisor: A/P ilya sergey)

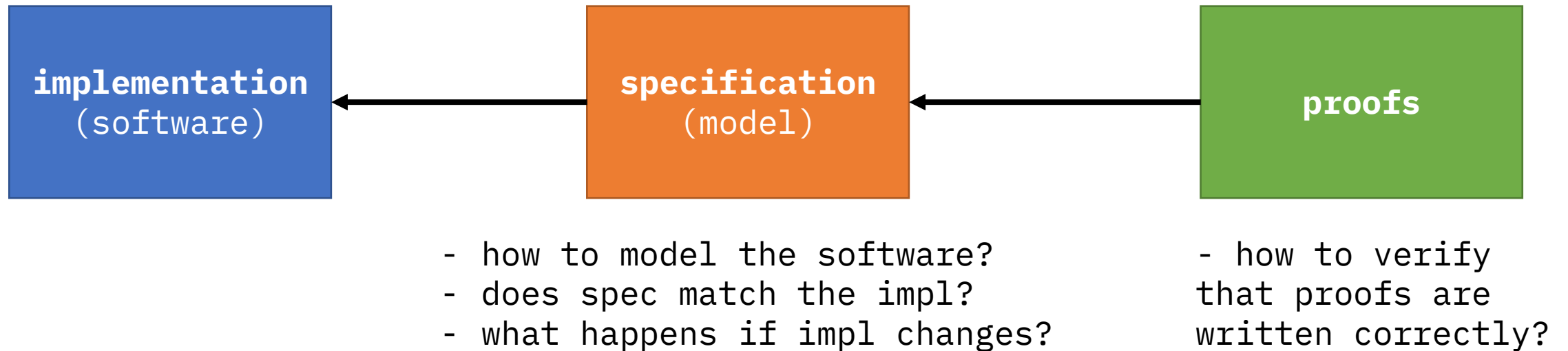
consensus

- fundamental problem in distributed computing
- **problem:** requires multiple different computing processes to agree on a single value
- **applications:** distributed databases, blockchain, cloud computing, etc
- **consensus protocols** are algorithms that enable processes to achieve consensus (i.e. agree on a single value) under certain conditions

correctness and formal verification

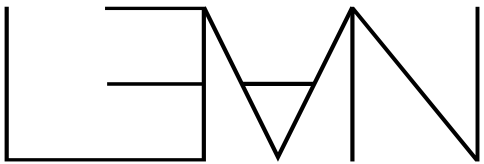
- consensus algorithms are pretty important, so we want to make sure they are implemented **correctly**
- **formal verification (of software)**: mathematically prove that a piece of software satisfies certain properties
- **limitations**: time/cost/effort, modelling drift, scalability, etc

correctness and formal verification



interactive theorem provers

- tools that enable users to construct and verify proofs using a mathematical framework
 - oftentimes are also implemented as programming languages themselves, due to Curry-Howard Correspondence



The Coq Proof Assistant



interactive theorem provers

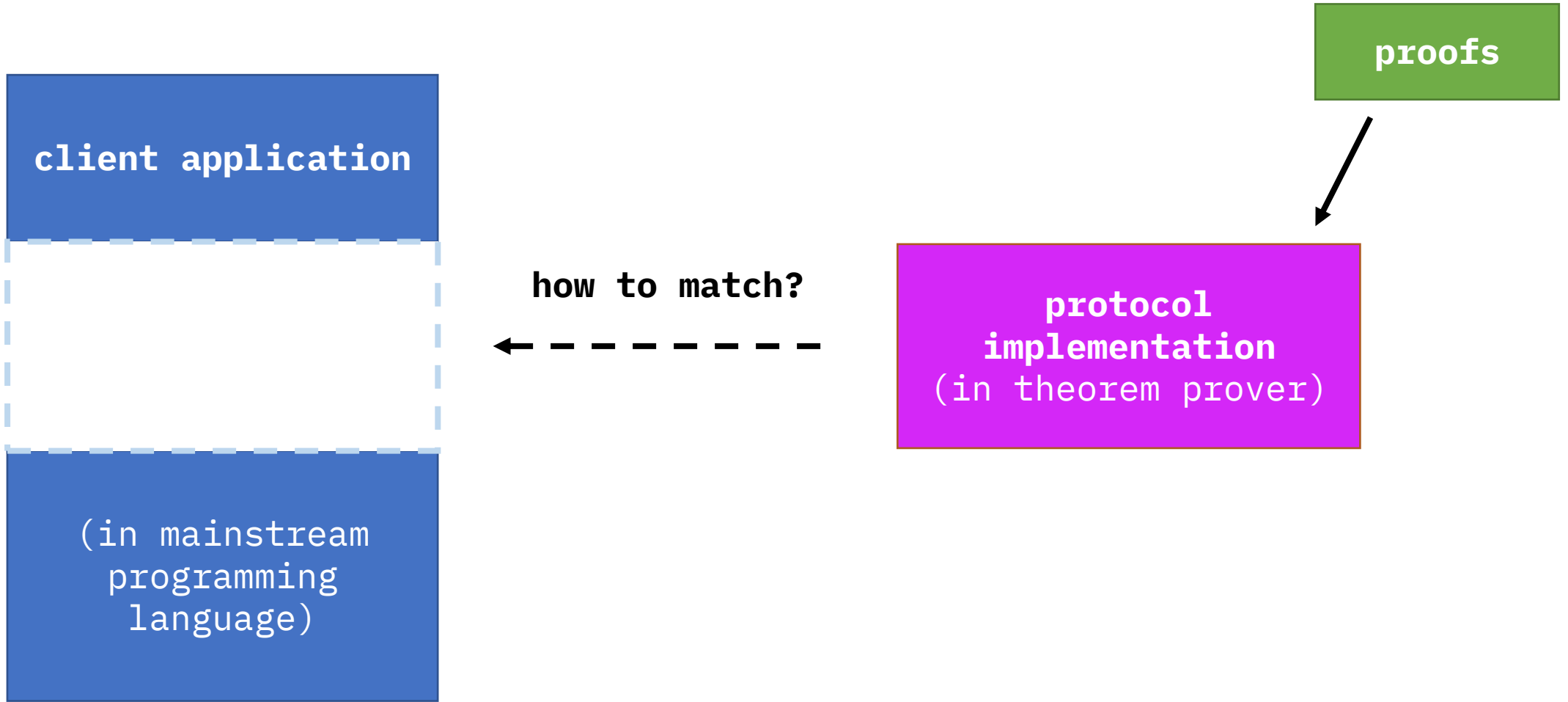
- **benefits:**

- write software implementation and proofs in the same language (fewer moving parts)
- computer-aided proof verification (if your proof compiles, it is verifiably correct within the logical framework of the prover)

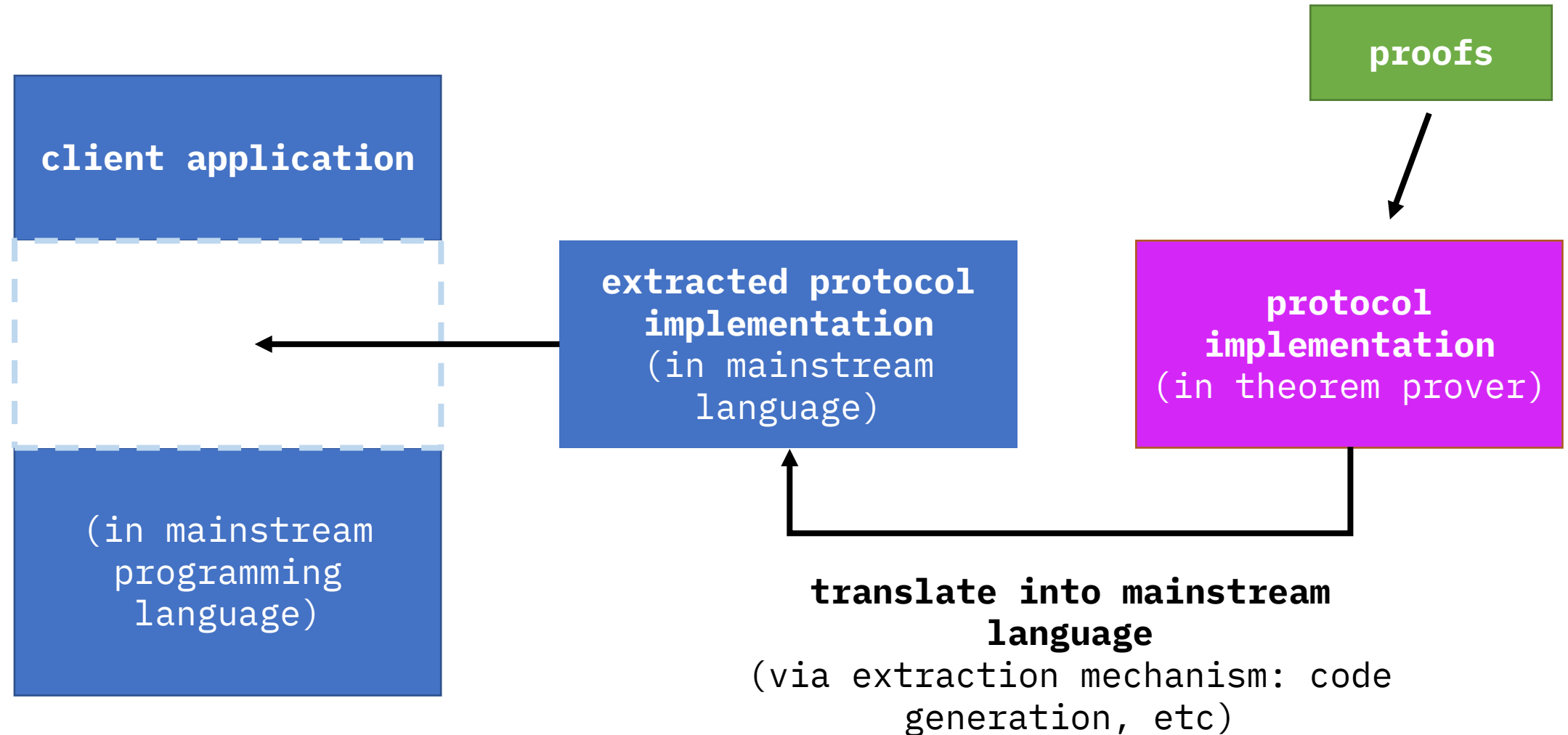
- **limitations:**

- may not be very practical for production use

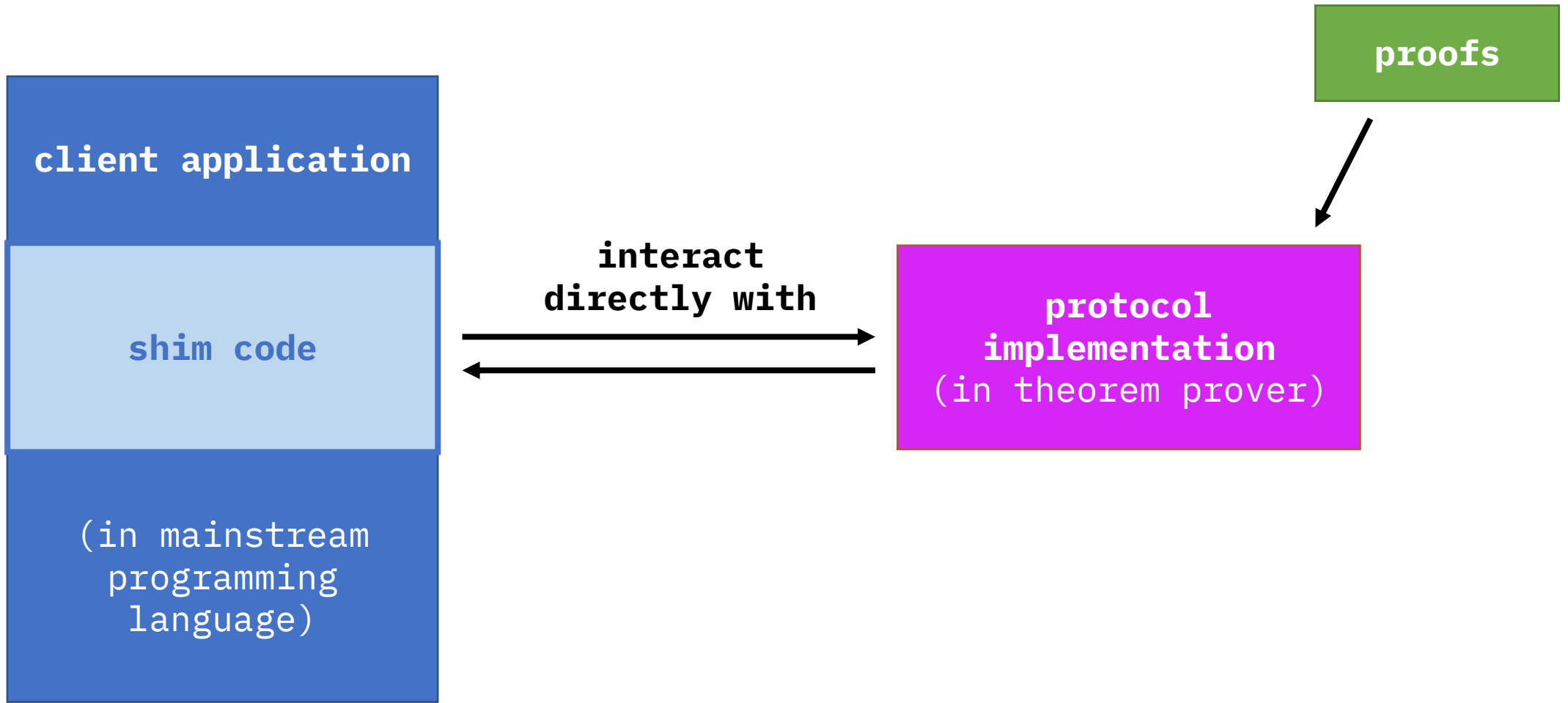
interactive theorem provers



program extraction

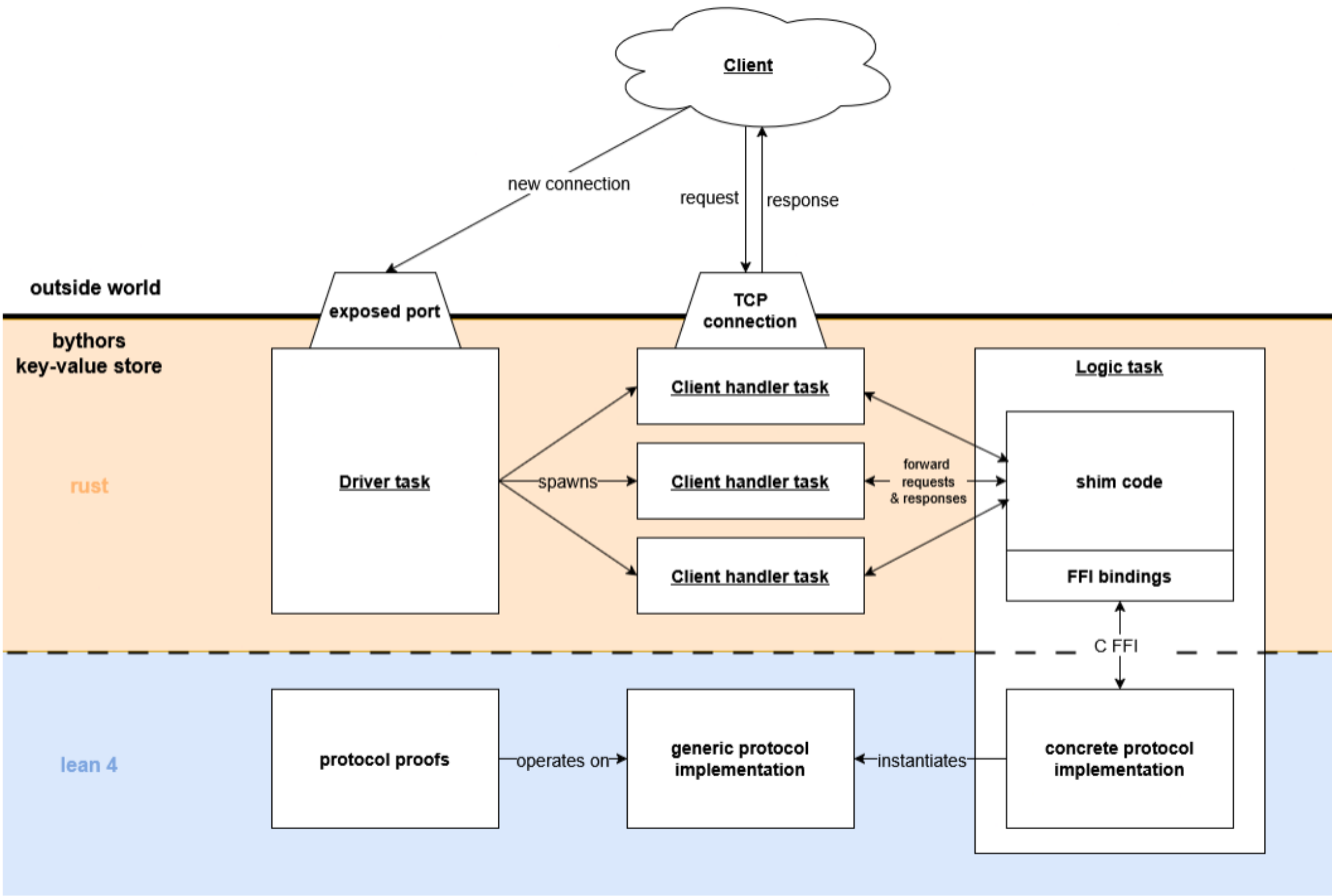


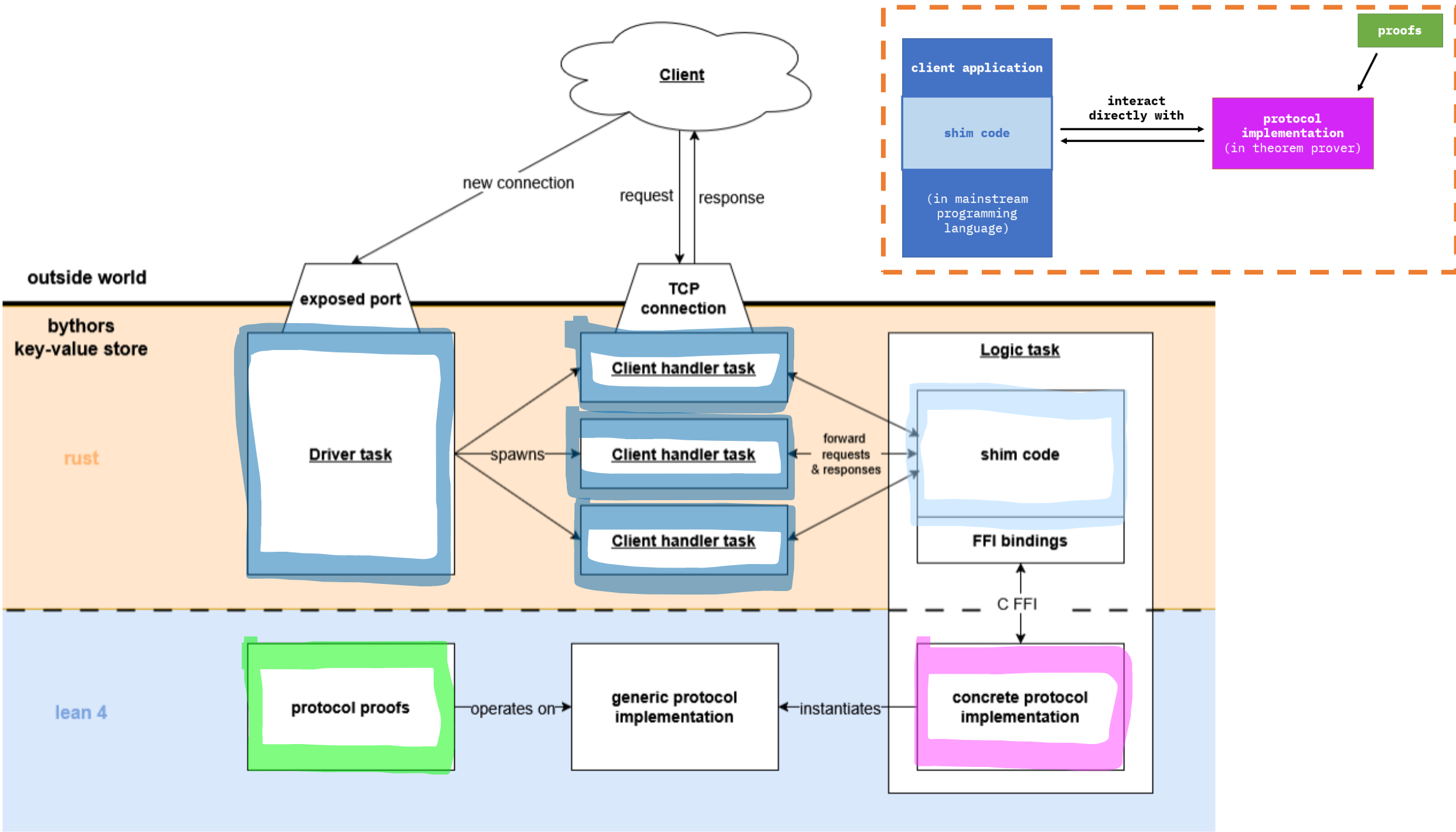
our approach: direct interaction



project overview

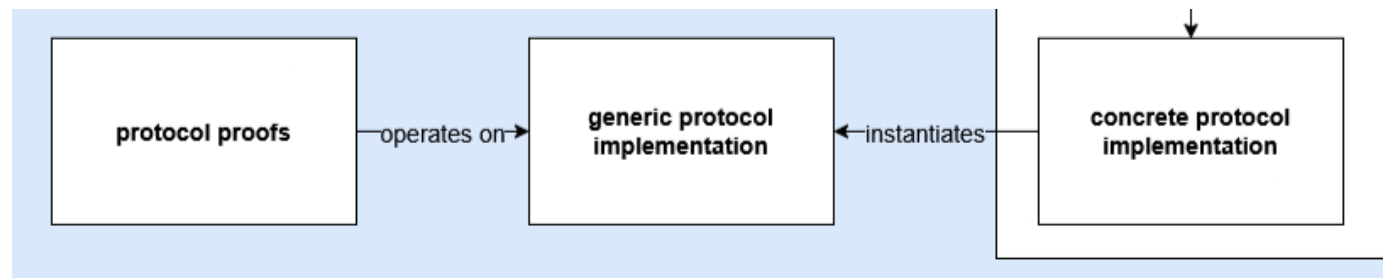
- apply a “direct interaction” approach in the context of a `verified` implementation of the Raft Consensus Protocol
- build a client application (replicated key-value store) using this approach, as a proof of concept
- **details:**
 - raft protocol implemented in the lean 4 theorem prover
 - shim code and client app implemented in rust





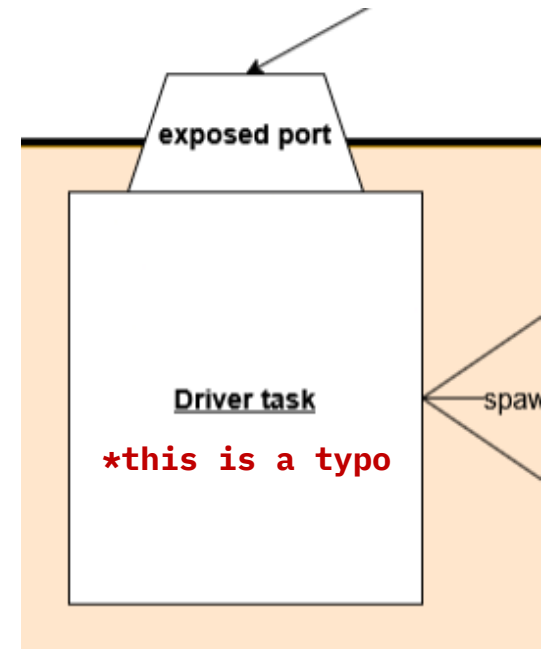
protocol implementation

- implementation of the Raft consensus algorithm in Lean
 - translation of Verdi-Raft, originally written in Coq
- generic implementation has type parameters
(e.g. for node address type, state machine type, message payload type, etc)
- (proofs not actually included, as they are out of scope for this project)



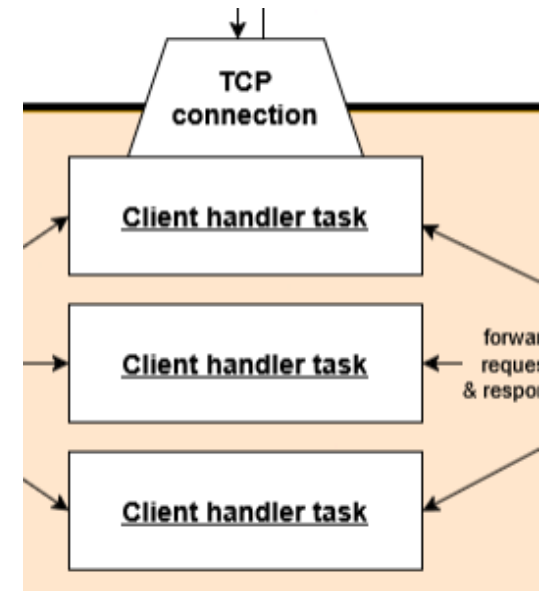
network task

- accepts TCP connections from clients
- sets up event streams for each client
- uses tokio library for networking



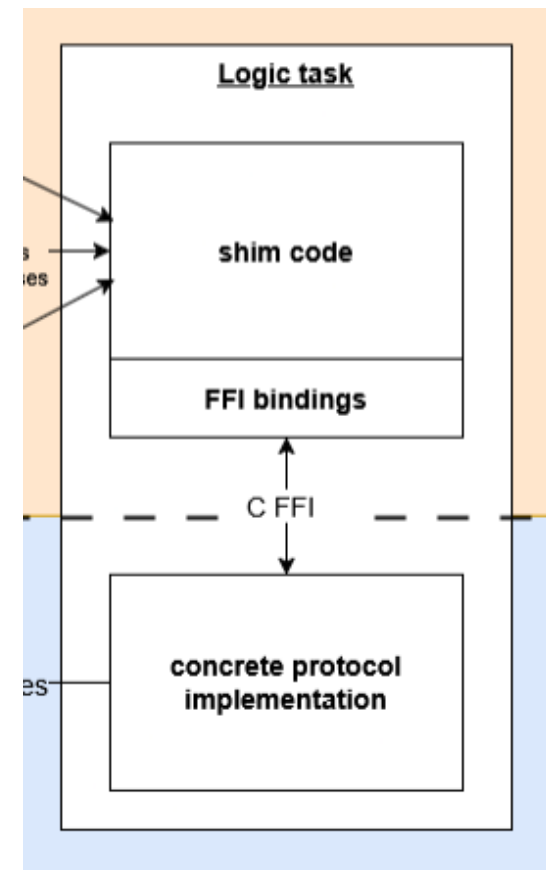
client handler task(s)

- processes requests and responses for a specific client (via a stream of events)
- forwards requests from client to logic task
- forwards responses from logic task back to client



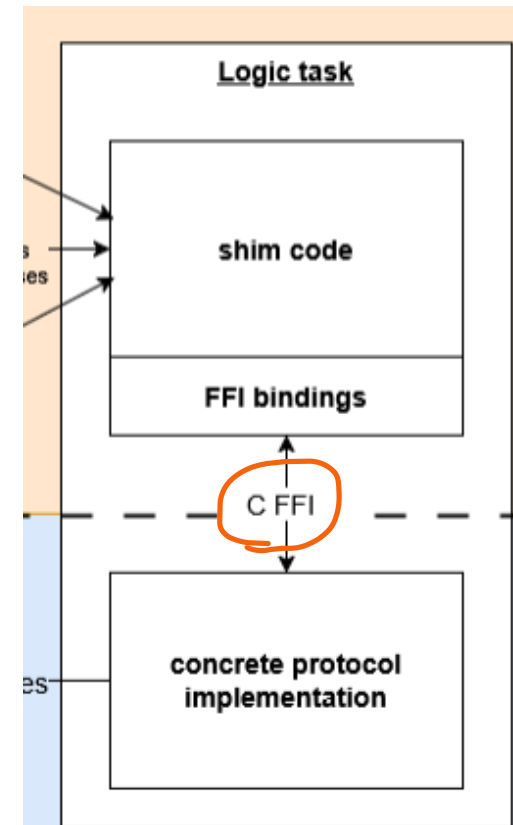
logic task

- FFI bindings allows us to call Lean 4 code from within Rust
- forwards client requests to the actual protocol logic in Lean
- communicates with other nodes in the network to reach consensus
- responsible for driving the protocol forward (e.g. sending heartbeats)



C FFI

- use the C foreign function interface (FFI) for Lean-Rust interoperation
 - **export** lean functions as C-callable functions
 - bind rust functions to **external** C declarations
 - **result**: calling lean functions from rust
- **complications**:
 - need to manage lean memory from rust (manually manage reference count of lean objects)
 - need to marshal data between lean and rust



performance benchmarks

- our project so far has been focused entirely on the notion of practicality
- demonstrate that our approach is indeed practically feasible by showing that it achieves comparable performance to other related work
 - **etcd**, a popular replicated key-value stores (unverified)
 - **vard**, a replicated key-value store built on top of the verified Verdi-Raft protocol
 - **Ivy-Raft**, a replicated key-value store built on top of a Raft protocol implemented and verified in Ivy

benchmarking procedure

- initialize each replicated key-value store with 3 nodes (i.e. 3 replicas)
- spawn 8 threads to send requests
- **in each thread:**
 - send 100 total requests, ~50/50 random split of GET and PUT reqs
 - keys are uniformly selected from a set of 50
 - measure time taken between sending request and receiving response

benchmarking results

Key-Value Store	Total Time (s)	Throughput (req/s)
Bythors	5.276967	151.602235
etcd	2.168403	368.935126
vard	14.442923	55.390450
Ivy-Raft *	0.108757	7355.847071

Table 4.1: Performance of SMR-based key-value stores

(*not a meaningful comparison, because they don't persist node state to disk)

future work

- patch potential memory leaks
(programmer error from low level memory management)
- further performance optimizations
(significant overhead when marshalling between lean and rust data representations)
- implement as a library for modularity
(plug-and-play different protocol implementations)

the end

thank you